~~Cigarette smokers~~ problem Spider-mans

Sommaire

Rappelle du problème....2
Contexte....2
Fonctionnement....2
Technique....2
Lore....2
Algo Thread-Safe....3
Algo Non Thread-Safe....5
Deadlock....6
Utilité informatique du problème....6

Rappelle du problème

Contexte

Il y a un agent et trois fumeurs. Chaque fumeur possède un type d'ingrédient (le fumeur A a du papier, le fumeur B a des filtres, le Fumeur C a des allumettes), mais il leur manque les autres ingrédients pour rouler une cigarette. L'agent possède les trois ingrédients manquants.

Fonctionnement

L'agent place aléatoirement sur la table deux des trois ingrédients.
Le fumeur qui possède les ingrédients complémentaires à ceux posés sur la table se réveille, récupère les ingrédients et fume sa cigarette.
Une fois la cigarette terminée, le fumeur signale à l'agent qu'il est prêt pour une nouvelle fourniture.

Technique

Le problème illustre la difficulté de coordonner des processus basés sur des conditions multiples : le fumeur ne peut agir que si les ingrédients posés est précisément ceux dont il a besoin, et non celui qu'il possède déjà.

Lore

Sur la Terre-928, Peter Parker est chargé de surveiller que chacun des nouveaux Spider-Man et tout le matériel nécessaire pour partir en mission (une tenue, des toiles et un Spider-bot). Chacun d’entre eux a constamment un outil avec lui (spider-Gwen a toujours des toiles, Spider-Punk a toujours une tenue et Miles Morales a toujours des spider-bots) on considère qu’ils sont une quantité infinie de chaque matérielle Peter Parker lui a une quantité infinie de chaque outil. Lorsqu'ils partent en mission, Peter Parker fournit 2 équipements en simultané (par exemple : des toiles et une tenue). Une fois que le Spiderman a récupéré ses équipements, il part en mission. Une fois celle-ci finie, ils reviennent voir Peter Parker.

Algo Thread-Safe

Variable Globale

```
SÉMAPHORES :
   peterParker = 1
   spiderWeb = 0, suit = 0, spiderBot = 0
   spiderWebGwen = 0, suitSpiderPunk = 0, spiderBotMilles = 0

BOOLEENS :
   is_SpiderWeb = faux, is_Suit = faux, is_SpiderBot = faux

MUTEX :
   pusher_mutex
```

Procédure de PeterParker (l’agent)

```
Procédure peterParkerLogic()
   Pour i allant de 0 à 9 :
      Attendre (peterParker)
      randInt = nombre aléatoire entre 0 et 2
      Si randInt == 0 :
         Afficher "Peter Parker met à disposition des toiles et des spider-bots"
         Signaler (spiderWeb)
         Signaler (spiderBot)
      Sinon Si randInt == 1 :
         Afficher "Peter Parker met à disposition des spider-bots et des tenues"
         Signaler (spiderBot)
         Signaler (suit)
      Sinon :
         Afficher "Peter Parker met à disposition des toiles et des tenues"
         Signaler (spiderWeb)
         Signaler (suit)
   Fin Pour
Fin Procédure
```

Procédure d'un des pushers (Il y en a une pour chaque éléments)

```
Procédure spiderWeb_pusher()
   Tant que (Vrai) :
      Attendre (spiderWeb)
      Verrouiller (pusher_mutex)
      Si is_Suit est Vrai :
         is_Suit = Faux
         Signaler (spiderBotMilles)
      Sinon Si is_SpiderBot est Vrai :
         is_SpiderBot = Faux
         Signaler (suitSpiderPunk)
      Sinon :
         is_SpiderWeb = Vrai
      Déverrouiller (pusher_mutex)
   Fin Tant que
Fin Procédure
```

Procédure d'un des spiderman en mission (il y en a autant que de spiderman (smokers))

```
Procédure spiderWebLogic()
   Tant que (Vrai) :
      Attendre (spiderWebGwen)
      Afficher "Spider-Gwen a ses toiles et récupère sa tenue et son spider-bot"
      Afficher "Elle part en mission..."
      Attendre (certain temps)
      Signaler (peterParker)
   Fin Tant que
Fin Procédure
```

Main

```
Procédure Main()
   Afficher "--- Début de la simulation ---"

   Lancer Thread thSpiderMan1(spiderWebLogic)
   Lancer Thread thSpiderMan2(suitLogic)
   Lancer Thread thSpiderMan3(spiderBotLogic)
   Lancer Thread thPusher1(spiderWeb_pusher)
   Lancer Thread thPusher2(suit_pusher)
   Lancer Thread thPusher3(spiderBot_pusher)
   Lancer Thread thPeterParker(peterParkerLogic)

   Attendre la fin du Thread thPeterParker (join)

   Afficher "--- Fin de la simulation ---"
Fin Procédure
```

Algo Non Thread-Safe

Variable global
SEMAPHORES:
petterParker = 1
spiderWeb = 0
suit = 0
spiderBot = 0

MUTEX :
pusher_mutex

Procédure de PeterParker (l'agent)

```
Procédure peterParkerLogic()
    Tant que(vraie)
        Attendre (peterParker)
        randInt = nombre aléatoire entre 0 et 2
        Si randInt == 0 :
            Afficher "Peter Parker met à disposition des toiles et des spider-bots"
            Signaler (spiderWeb)
            Signaler (spiderBot)
        Sinon Si randInt == 1 :
            Afficher "Peter Parker met à disposition des spider-bots et des tenues"
            Signaler (spiderBot)
            Signaler (suit)
        Sinon :
            Afficher "Peter Parker met à disposition des toiles et des tenues"
            Signaler (spiderWeb)
            Signaler (suit)
    Fin Pour
Fin Procédure
```

Procédure d'un des spiderman en mission (il y en a autant que de spiderman (smokers))

```
Procédure spiderWebLogic()
    Tant que (Vrai) :
        Attendre (suit)
        Attendre (spiderBot)
        Afficher "Spider-Gwen a ses toiles et récupère sa tenue et son spider-bot"
        Afficher "Elle part en mission..."
        Signaler (peterParker)
    Fin Tant que
Fin Procédure
```

Main

```
Procédure Main()
   Afficher "--- Début de la simulation ---"

   Lancer Thread thSpiderMan1(spiderWebLogic)
   Lancer Thread thSpiderMan2(suitLogic)
   Lancer Thread thSpiderMan3(spiderBotLogic)
   Lancer Thread thPeterParker(peterParkerLogic)

   Attendre la fin du Thread thPeterParker (join)

   Afficher "--- Fin de la simulation ---"
Fin Procédure
```

Deadlock

Dans notre programme, le dead lock est dû au fait que chaque spiderman récupère ses 2 ressources manquantes l'une après l'autre au lieu des 2 en même temps comme dans le programme safe.

Ce que ça fait, c'est que si, par exemple, Peter Parker met à disposition des toiles et une tenue, il sort en premier les toiles. Spider-Punk, qui a besoin des toiles, les prend, mais Gwen a besoin de la tenue, donc elle la prend et les 2 attendent leur prochaine ressource. Or le thread ne peut pas se finir dans ces conditions, ce qui crée un dead lock.

Utilité informatique du problème

Le "cigarette smoker problem" est une allusion directe au fonctionnement de l'OS qui met à disposition des ressources pour que les programmes puissent les utiliser. En effet, l'OS est représenté par l'agent, tandis que les fumeurs sont représentés par les programmes. Dans le pseudo-code, les pusher représentent les fonctions intermédiaires de l'OS, ajoutant les ressources brutes fournies par l'OS pour mettre à jour la table des états globaux du système et vérifier la liste de processus en attente.